Documentación Final de los Proyectos de Estructuras de Datos

Marco Antonio Gómez Fuentes, Emiliano Peniche Correa,

Amir Moisés Hernández Farah y Rodrigo Quijano López

Equipo ProyectoFinal 7

Ingeniería en Tecnologías de Información y Negocios Digitales, Universidad Anáhuac Mayab

Estructuras de Datos y Algoritmos (202660_50560)

Mtra. Lizbeth Hernández Olan

10 de noviembre de 2026

Documentación Final de los Proyectos de Estructuras de Datos

Este documento junta, en orden, la documentación de los proyectos que hicimos en la materia. Cada proyecto se resume con las cinco fases de la plantilla: contextualización, requerimientos funcionales, diseño, codificación y pruebas. Para que no quedara tan largo, se dejaron solo los datos más importantes, el diagrama de clases y una o dos capturas de cada uno. La documentación completa y el código están en el repositorio del equipo.

Para la entrega final, todos los proyectos se van a presentar en modo gráfico. La Tabla 1 muestra cómo va cada uno.

Tabla 1

Estado de los Proyectos

Proyecto	**Lenguaje**	**Estructura**	**Estado**
1. Resistencias eléctricas	Python	Arreglos	Terminado, con modo gráfico
2. Carrito de compras	Python	Listas	Terminado, con modo gráfico
3. Navegador web	Python	Pilas	Terminado en consola; modo gráfico en avance
4. Cola de impresión	Python	Colas	Terminado en consola
5. Sistema de archivos	Java	Árboles (ABB)	En avance
6. Ruta mágica	—	—	Por iniciar; se agregará en la siguiente versión

Proyecto 1. Resistencias Eléctricas

Contextualización

El programa traduce el valor de una resistencia en ohmios a los colores de sus bandas y calcula el valor total de n resistencias conectadas en serie y en paralelo. Las n resistencias reciben un valor entero aleatorio entre 10 y 1 000 000 000 Ω y se guardan en un arreglo. Se usan la tabla del código de colores de 4 bandas y las fórmulas Rts = R1 + R2 + … + Rn y 1/Rtp = 1/R1 + 1/R2 + … + 1/Rn.

Requerimientos Funcionales

Tabla 2

Requerimientos del Proyecto 1

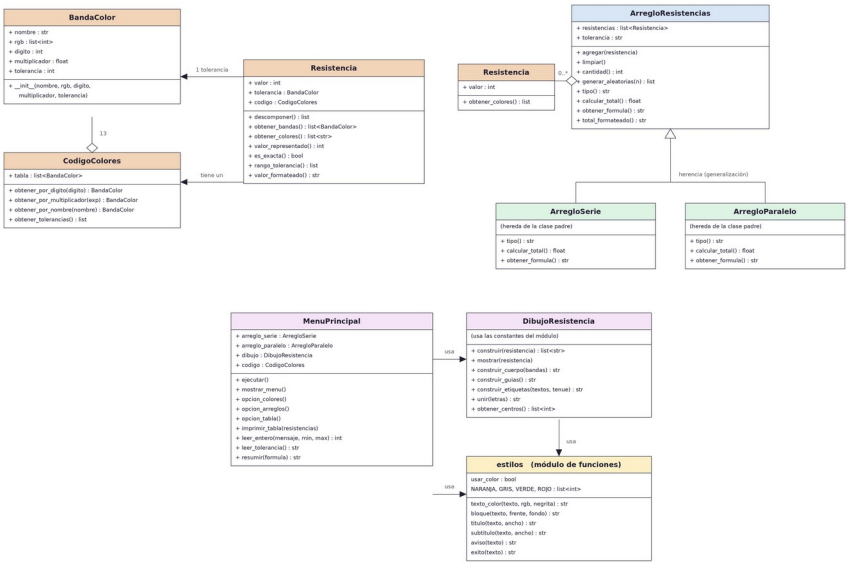
Clave	Requerimiento	Entrada → Salida
R1	Determinar los colores de una resistencia	Valor en Ω y tolerancia → colores de las 3 bandas (160 Ω: Marrón, Azul, Marrón)
R2	Calcular el total en serie	Arreglo de n resistencias → Rts en Ω, kΩ, MΩ o GΩ
R3	Calcular el total en paralelo	Arreglo de n resistencias → Rtp en Ω, kΩ, MΩ o GΩ
R4	Generar n resistencias aleatorias	Entero n ≥ 1 → arreglo con n resistencias
R5	Mostrar la resistencia en modo gráfico	Resistencia → dibujo con sus 4 bandas de color
R6	Validar los datos capturados	Lo que escribe el usuario → aviso o continuación

Diseño

El sistema quedó en ocho clases. BandaColor, CodigoColores y Resistencia resuelven el código de colores. ArregloResistencias es la clase padre que guarda el arreglo y lo llena al azar; ArregloSerie y ArregloParalelo heredan de ella y solo cambian calcular_total() con su fórmula (Figura 1).

Figura 1

Diagrama de Clases del Proyecto 1



Codificación

Se programó en Python 3, una clase por archivo. Lo más importante de R1 es el método descomponer(), que separa el valor en dos cifras significativas y el exponente del multiplicador (por ejemplo, 5800 = 58 × 10^2, que da Verde, Gris y Rojo). Para la entrega final se agregó ventana_principal.py, una ventana hecha con Tkinter que reutiliza las mismas clases y tiene tres pestañas: colores, serie y paralelo, y tabla de colores (Figura 2).

Figura 2

Modo Gráfico: Colores de una Resistencia



Pruebas

Tabla 3

Pruebas del Proyecto 1

#	Entrada	Salida obtenida	Resultado
1	R1: 160 Ω	Marrón, Azul, Marrón	Correcto
2	R1: 5800 Ω	Verde, Gris, Rojo	Correcto
3	R1: 10 Ω y 1 000 000 000 Ω (límites)	Marrón, Negro, Negro / Marrón, Negro, Gris	Correcto
4	R2: 100, 220 y 330 Ω en serie	650 Ω	Correcto
5	R3: 100, 220 y 330 Ω en paralelo	56.8966 Ω	Correcto
6	R6: 9 Ω, el texto abc y n = 0	Mensaje de advertencia	Correcto

Estos casos también están en pruebas.py, que termina con 37 de 37 pruebas correctas. Usar herencia nos permitió escribir una sola vez el manejo del arreglo; agregar otro tipo de conexión solo necesitaría otra clase hija.

Proyecto 2. Carrito de Compras

Contextualización

El programa simula el carrito de compras de una tienda con un catálogo precargado de cinco productos (Teclado $450, Mouse $250, Monitor $3200, Audífonos $800 y Memoria USB $180). Toda la información vive en memoria dentro de listas: una para el catálogo y otra para el carrito, sobre las que se hacen inserciones, búsquedas, actualizaciones y eliminaciones.

Requerimientos Funcionales

Tabla 4

Requerimientos del Proyecto 2

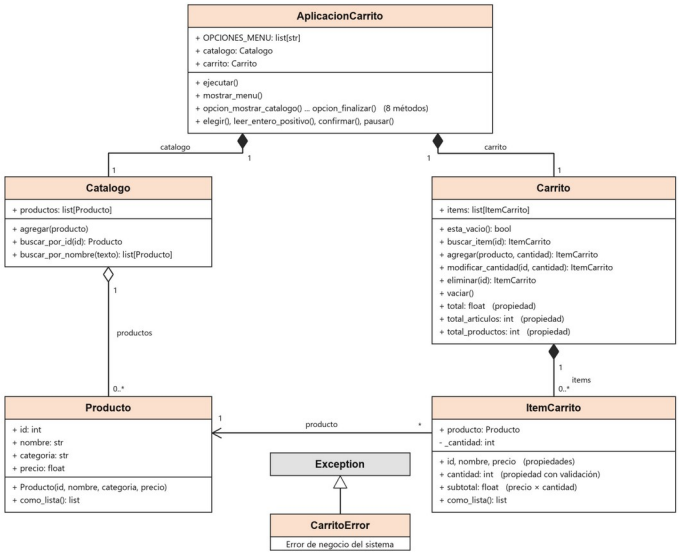
Clave	Requerimiento	Entrada → Salida
R1	Mostrar catálogo	Opción → tabla con ID, nombre, categoría y precio
R2	Buscar producto por ID o nombre	ID o texto → producto(s) encontrados
R3	Agregar producto al carrito	Producto y cantidad → carrito actualizado
R4	Ver carrito	Opción → productos, subtotales y total
R5	Modificar cantidad	Producto y nueva cantidad → subtotal actualizado
R6	Eliminar producto	ID o nombre → producto quitado del carrito
R7	Vaciar carrito	Confirmación → carrito vacío
R8	Finalizar compra	Opción → resumen, artículos y total a pagar
R9	Menú principal y salir	Opción → operación elegida o fin del programa

Diseño

Producto guarda los datos de cada artículo; ItemCarrito es un renglón del carrito con un producto y su cantidad; Catalogo y Carrito son las listas. CarritoError es la excepción propia que se usa para los errores de la tienda (Figura 3).

Figura 3

Diagrama de Clases del Proyecto 2



Codificación

El código se separó en utilidades.py, modelo.py (las clases, sin print ni input) y carrito_compras.py (consola). Para la entrega final se agregó ventana_carrito.py, una ventana con Tkinter con el catálogo de un lado y el carrito del otro, que llama a los mismos métodos del modelo (Figura 4). En la revisión con el asistente de IA se corrigió un error: agregar una cantidad negativa a un producto que ya estaba en el carrito le restaba unidades; ahora se valida antes de sumar.

Figura 4

Modo Gráfico: Catálogo y Carrito



Pruebas

Tabla 5

Pruebas del Proyecto 2

#	Entrada	Salida obtenida	Resultado
1	R2: buscar "audifonos" sin acento	Encuentra Audífonos (ID 4)	Correcto
2	R3: 2 Teclados y 1 Audífonos	Total $1,700	Correcto
3	R3: agregar otro Teclado	Suma la cantidad, no crea otro renglón	Correcto
4	R3: cantidad −1	Mensaje de error, la cantidad no cambia	Correcto
5	R5: cantidad 0	Mensaje de error	Correcto
6	R7: vaciar carrito	Carrito vacío, total $0	Correcto

El archivo pruebas.py termina con 28 de 28 pruebas correctas. Las propiedades calculadas (subtotal y total) hacen que los importes se actualicen solos cada vez que cambia el carrito.

Proyecto 3. Navegador Web

Contextualización

El programa imita el historial de un navegador con dos pilas (LIFO). La pila de atrás guarda las páginas por las que ya pasó el usuario y la pila de adelante las páginas a las que puede volver después de un Back. Al visitar una página nueva, la actual se apila atrás y la pila de adelante se vacía; con Back y Forward las páginas pasan de una pila a la otra.

Requerimientos Funcionales

Tabla 6

Requerimientos del Proyecto 3

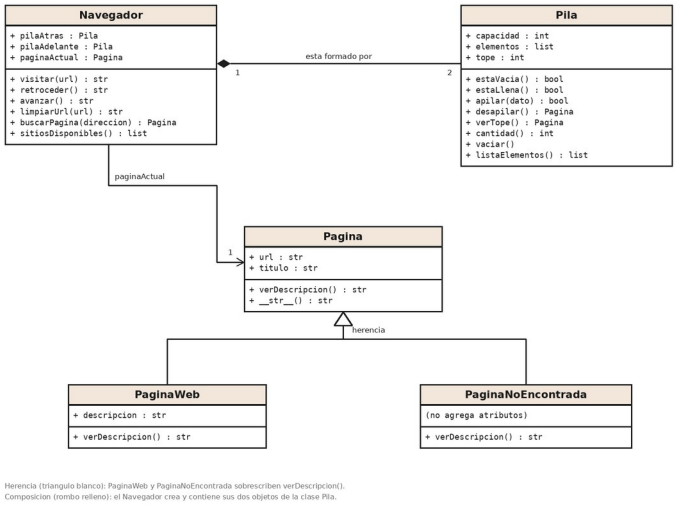
Clave	Requerimiento	Entrada → Salida
R1	Visitar una página nueva	URL → página actual; la pila de atrás crece
R2	Regresar (Back)	Opción 2 → página anterior
R3	Avanzar (Forward)	Opción 3 → página siguiente
R4	Mostrar la página actual	Cualquier operación → página y las dos pilas
R5	Salir (Exit)	Opción 4 → fin del programa
R6	Evitar errores sin páginas	Pila vacía o llena → aviso PILA VACIA / PILA LLENA
R7	Limpiar la pila de avance	Visitar página nueva → pila de adelante vacía
R8	Mensajes claros	URL desconocida → página 404 y mensaje

Diseño

La clase Pila se hizo con un arreglo y un índice tope, con capacidad de cuatro páginas. PaginaWeb y PaginaNoEncontrada heredan de Pagina, así la pila guarda las dos sin distinguirlas. Navegador contiene las dos pilas y aplica las reglas (Figura 5).

Figura 5

Diagrama de Clases del Proyecto 3



Codificación

El código está en pila.py, pagina.py, navegador.py y main.py (menú de consola). Apilar y desapilar revisan primero si la pila está llena o vacía, así nunca hay desbordamiento. Como avance del modo gráfico se agregó interfaz.py, una ventana con Tkinter que usa la misma clase Navegador: ya permite escribir o elegir una URL y visitarla (Figura 6). Falta conectar los botones de atrás y adelante y dibujar las pilas.

Figura 6

Avance del Modo Gráfico del Navegador



Pruebas

Tabla 7

Pruebas del Proyecto 3

#	Entrada	Salida obtenida	Resultado
1	Visitar cuatro páginas seguidas	Quedan en la pila de atrás en orden LIFO	Correcto
2	Back con historial	Regresa y la actual pasa a la pila de adelante	Correcto
3	Forward después de un Back	Avanza y la actual regresa atrás	Correcto
4	Back o Forward con la pila vacía	Mensaje PILA VACIA	Correcto
5	Visitar con la pila de atrás llena	Mensaje PILA LLENA, no cambia de página	Correcto
6	URL fuera del catálogo	Página de error 404	Correcto

Figura 7

Versión de Consola: Back con Historial Disponible



Lo que más nos costó fue entender por qué la pila de adelante se vacía al visitar una página nueva. Ponerle un límite de cuatro páginas a cada pila nos dejó ver qué pasa cuando se llena, algo que con una lista normal de Python no se notaría.

Proyecto 4. Cola de Impresión

Contextualización

El programa simula una impresora compartida por cinco computadoras (PC1 a PC5). Cada trabajo que llega se guarda en una cola y se atiende con la regla FIFO: el primero que entra es el primero que se imprime. La cola tiene capacidad para seis trabajos y la computadora que manda cada trabajo se elige al azar.

Requerimientos Funcionales

Tabla 8

Requerimientos del Proyecto 4

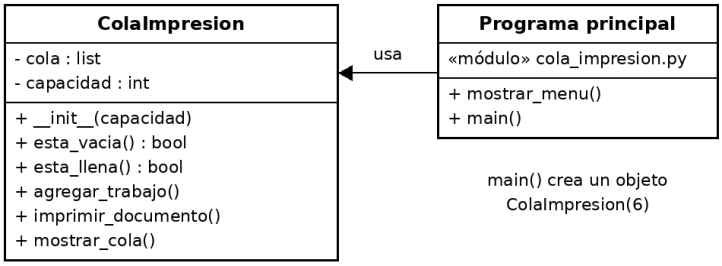
Clave	Requerimiento	Entrada → Salida
R1	Agregar trabajo de impresión	Opción 1 → trabajo de una PC al azar al final de la cola
R2	Imprimir documento	Opción 2 → se atiende y se quita el primero de la cola
R3	Mostrar cola	Opción 3 → trabajos en el orden en que se atenderán
R4	Verificar cola llena	6 trabajos → mensaje "La cola de impresión está LLENA"
R5	Verificar cola vacía	0 trabajos → mensaje "La cola de impresión está VACÍA"
R6	Mostrar menú	Inicio o fin de una operación → menú de 4 opciones
R7	Salir	Opción 4 → "Programa finalizado"

Diseño

El sistema tiene una clase, ColaImpresion, con la lista de trabajos y la capacidad máxima. Sus métodos revisan si la cola está vacía o llena, agregan, imprimen y muestran los trabajos. Las funciones mostrar_menu() y main() forman el programa principal, que crea una cola de capacidad seis (Figura 8).

Figura 8

Diagrama de Clases del Proyecto 4



Codificación

Se programó en Python en el archivo cola_impresion.py. La cola es una lista: los trabajos entran al final con append() y salen del inicio con pop(0), y así se respeta FIFO. La librería random elige la computadora con random.choice(), los condicionales controlan el menú y los casos de cola llena o vacía, y un ciclo while mantiene el programa abierto hasta que el usuario elige salir.

Pruebas

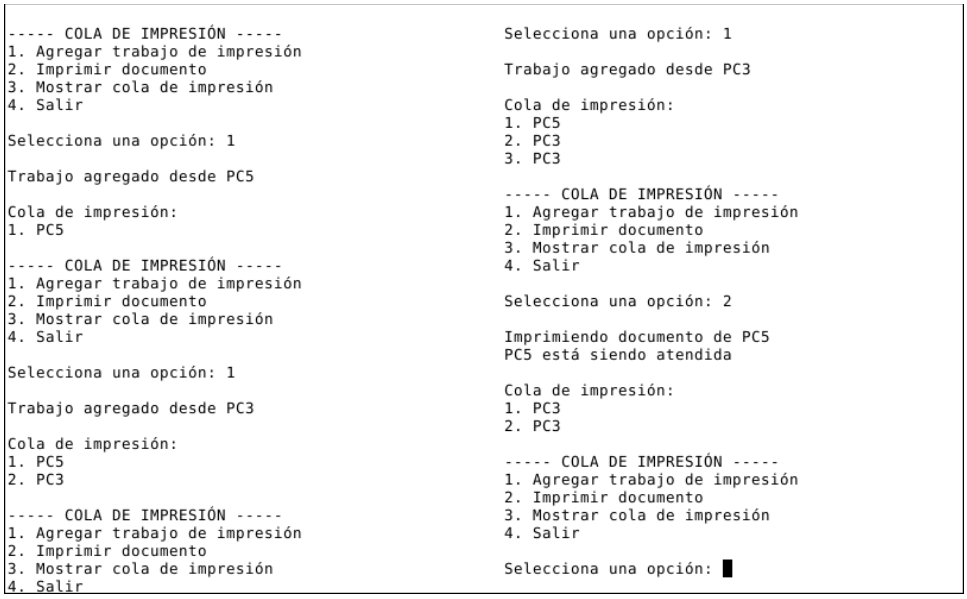
Tabla 9

Pruebas del Proyecto 4

#	Entrada	Salida obtenida	Resultado
1	Imprimir con la cola vacía	"La cola de impresión está VACÍA"	Correcto
2	Agregar tres trabajos	Se guardan en orden de llegada	Correcto
3	Imprimir con tres trabajos	Se atiende el primero (FIFO)	Correcto
4	Agregar un séptimo trabajo	"La cola de impresión está LLENA"	Correcto
5	Opción 8	"Opción no válida" y vuelve el menú	Correcto
6	Opción 4	"Programa finalizado"	Correcto

Figura 9

Ejecución: Tres Trabajos Agregados y Atención FIFO



Nota. Se agregan trabajos de PC5, PC3 y PC3; al imprimir se atiende primero PC5, que fue el primero en llegar.

La cola fue la estructura más sencilla de los proyectos, pero nos ayudó a ver la diferencia con la pila: aquí el primero que entra es el primero que sale.

Proyecto 5. Sistema de Archivos con Árbol Binario de Búsqueda

Contextualización

El programa, hecho en Java, simula un sistema de archivos con un árbol binario de búsqueda (ABB). Cada nodo es un directorio o un archivo y se ordena alfabéticamente por su nombre: los menores a la izquierda y los mayores a la derecha. La raíz siempre es un directorio y no se puede eliminar, y un archivo siempre es hoja porque no puede contener otros elementos.

Requerimientos Funcionales

Tabla 10

Requerimientos del Proyecto 5

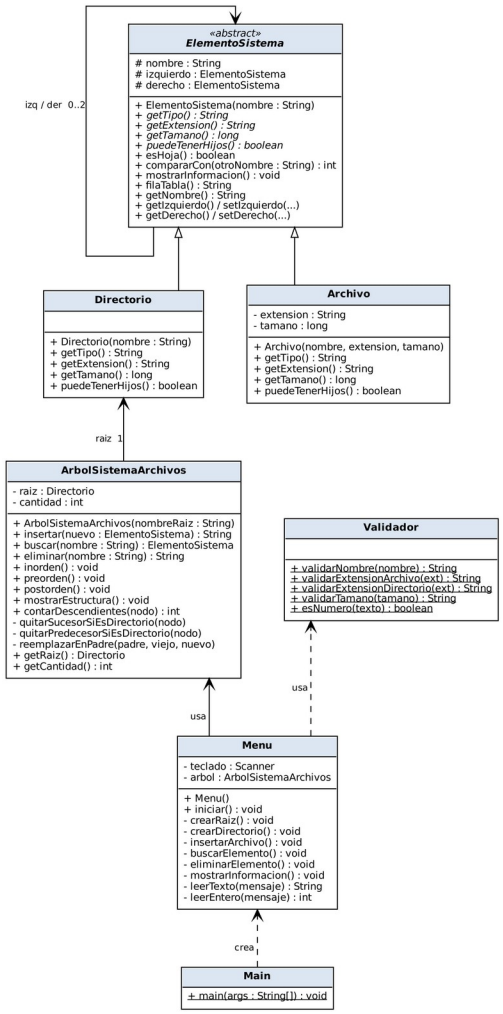
Clave	Requerimiento	Entrada → Salida
R1	Crear directorio raíz	Nombre → árbol con ese directorio como raíz
R2	Crear directorio	Nombre → nodo insertado a la izquierda o derecha
R3	Insertar archivo	Nombre, extensión y tamaño → archivo como hoja
R4	Buscar elemento	Nombre → datos del elemento o "no existe"
R5	Eliminar elemento	Nombre → se elimina (hoja, un hijo o dos hijos)
R6–R8	Recorridos inorden, preorden y postorden	Opción → tabla de nodos en ese orden
R9	Mostrar información	Nombre → datos, hijos y si es hoja
R10	Validar reglas	Datos → mensaje de error sin modificar el árbol
R11	Menú y salida	Opción 0–9 → operación o fin del programa

Diseño

ElementoSistema es la clase abstracta del nodo, con nombre, hijo izquierdo e hijo derecho; Directorio y Archivo heredan de ella. ArbolSistemaArchivos guarda la raíz y tiene las operaciones del ABB, Validador revisa los datos y Menu lee las opciones con Scanner (Figura 10).

Figura 10

Diagrama de Clases del Proyecto 5



Codificación y Estado

Cada clase está en su propio archivo .java. Ya funcionan crear la raíz, crear directorios, insertar archivos con sus validaciones y buscar. Para la entrega final faltan eliminar (con sus tres situaciones), los tres recorridos y mostrar información, además de las pruebas con capturas y el modo gráfico.

Referencias

Hernández Bejarano, M., y Baquero Rey, L. E. (2022). Estructuras de datos: Fundamentación práctica (1.ª ed.). RA-MA Editorial. https://elibro.net/es/ereader/mayab/230581

Hernández Olan, L. (2026a). *Documentación del proyecto* [Plantilla de entrega]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Hernández Olan, L. (2026b). *Java – Stack* [Material del módulo 4]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Hernández Olan, L. (2026c). *Navegador web: Proyecto de pila (LIFO – Last In, First Out)* [Especificación del proyecto]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Hernández Olan, L. (2026d). *Pilas* [Presentación del módulo 4]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Python Software Foundation. (2026). *tkinter: Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html